REINFORCEMENT LEARNING HOMEWORK 2

PROJECT 14

ROBOT NAVIGATION WITH SUB-GOALS USING HIERARCHICAL REINFORCEMENT LEARNING

Submitted by

Name	Roll Number
Shivani Bhasme	100523733004
C. Sreenithya	100523733010
P. Navya Sri	100523733044

Department: Computer Science and Engineering
Institution: University College of Engineering, Osmania University

Course: Reinforcement Learning
Submission date: 7 November 2026

A simulation-based project using Q-learning and a manager-worker navigation structure in a grid maze.

PROBLEM FORMULATION

Problem statement: Develop a simulated robot navigation system that learns to move through a grid maze, avoids obstacles, reaches an intermediate sub-goal when selected, and ultimately reaches the final destination.

Objective: Use Reinforcement Learning to learn suitable movement decisions. Organize navigation into smaller tasks using a high-level manager and low-level worker policies, then compare the learned agent with a random navigation baseline.

Environment: A fixed 5 × 5 grid maze represented using a NumPy array. The robot starts at position (0, 0), the intermediate sub-goal is at (4, 2), and the final goal is at (4, 4). Empty cells are traversable, while cells marked as obstacles cannot be entered. The environment is discrete and simulated.

Component	Definition used in this project
State (S)	The current position of the robot in the maze, represented by a pair (row, column).
Action (A)	Four discrete actions: Up, Down, Left and Right. The actions are encoded as 0, 1, 2 and 3 respectively.
Reward (R)	+100 when the selected target is reached; –10 for an invalid move (outside the grid or into an obstacle); and –1 for a normal movement. At the manager level, successful completion of the sub-goal option receives a step-adjusted bonus, final-goal completion receives a step-adjusted reward, and option failure receives –50.
Environment	The 5 × 5 grid maze containing free cells, obstacles, a start position, an intermediate sub-goal and a final destination.
Agent	The robot controller, consisting of a high-level manager and low-level worker policies.
Learning algorithm	Tabular Q-learning with an epsilon-greedy action-selection strategy. Worker Q-tables learn route-specific movement policies. A manager Q-table learns which navigation option to select.
Episode	One training or evaluation attempt. A worker episode ends when its target is reached or the step limit is reached.
Evaluation	Compare the hierarchical RL agent with a random navigation agent over 100 attempts. Measure successful attempts, success rate and average steps among successful attempts.

Reported experimental result: The random agent completed 32 out of 100 attempts (32%), while the hierarchical RL agent completed 100 out of 100 attempts (100%) in the tested maze and evaluation setup.

Scope: This project is a grid-world simulation. The reported result applies to the fixed maze and evaluation procedure used; it is not a claim of performance on unseen mazes or a physical robot.

1. ABSTRACT

This project studies robot navigation using Hierarchical Reinforcement Learning. A robot must move through a 5 × 5 maze, avoid obstacles and reach a destination. The navigation task is organized into options: a high-level manager selects an option, and a low-level worker policy chooses individual movements.

The notebook implements tabular Q-learning for three worker navigation skills and a Q-table for the manager. The evaluation compares the learned hierarchical agent with a random navigation baseline over 100 attempts. The random agent achieved 32% success, while the hierarchical RL agent achieved 100% success in the tested maze.

2. INTRODUCTION

Reinforcement Learning (RL) is a machine-learning approach in which an agent learns by interacting with an environment. It receives rewards for useful actions and penalties for undesirable actions. Through repeated interaction, the agent improves its decisions.

Robot navigation can involve many movement decisions. A hierarchical approach organizes a larger task into smaller sub-tasks, such as reaching an intermediate waypoint before proceeding to the final destination.

3. OBJECTIVES

• Represent robot navigation as a discrete grid-world problem.
• Implement Q-learning for movement decisions.
• Train separate worker policies for the selected navigation routes.
• Train a high-level manager to choose navigation options.
• Plot learning curves and success-rate graphs.
• Compare the learned agent with a random navigation baseline.

4. ENVIRONMENT DESCRIPTION

The environment is a 5 × 5 grid maze. In the notebook, 0 represents an empty cell, 1 represents an obstacle, 2 represents the start, 3 represents the final goal and 4 represents the sub-goal. The start is (0, 0), the sub-goal is (4, 2), and the final goal is (4, 4).

The robot can move Up, Down, Left or Right. If a movement goes outside the grid or enters an obstacle, the robot stays in its current state and receives a penalty.

5. SYSTEM ARCHITECTURE AND METHODOLOGY

High-level manager: The manager uses a 2 × 2 Q-table. At the start state, it can select either the sub-goal option or the direct-to-final-goal option. Once the sub-goal has been reached, the available manager action is to continue to the final goal.

Low-level workers: Three worker Q-tables are trained: Start → Sub-goal, Start → Final Goal, and Sub-goal → Final Goal. Each worker selects primitive movement actions using its learned Q-values.

Training process: The notebook first defines the maze and transition rules, then trains the worker policies, trains the manager over navigation options, evaluates the manager's selected option, runs the random baseline and plots the results.

6. Q-LEARNING ALGORITHM

Q-learning is a model-free reinforcement learning algorithm. It learns an action-value function Q(s,a), which estimates the expected future reward from taking action a in state s.

Update equation: $Q(s,a) \leftarrow Q(s,a) + \alpha\,[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$.

Here, α is the learning rate, γ is the discount factor, r is the received reward, s is the current state, and s′ is the next state. The notebook uses $\alpha = 0.1$ and $\gamma = 0.9$. Epsilon-greedy selection supports exploration and exploitation. Worker training runs for 3000 episodes, and manager training runs for 2000 episodes.

7. IMPLEMENTATION DETAILS

The project is implemented in Python in Google Colab. NumPy is used for the maze and Q-tables, Matplotlib is used for visualizations, and Python's random module is used for randomized action selection. The notebook sets Python and NumPy random seeds to 42.

The worker training function initializes a Q-table of shape (ROWS, COLS, 4), selects actions using epsilon-greedy exploration, calls the movement function, updates the Q-value and records episode reward and success. The manager trains over the pre-trained worker options.

8. EXECUTION RESULTS

The notebook output confirms that all libraries were imported, the maze was created, the movement rules and worker function were defined, all three worker skills were trained, and high-level manager training completed.

The manager Q-table printed by the notebook is: [[108.82, 99.2], [0.0, 99.8]]. The manager selected option 0, meaning it chose to reach the sub-goal first. The printed output confirms: Robot reached destination: True.

The evaluation output reports 32 successful random attempts out of 100 and 100 successful hierarchical RL attempts out of 100. Average successful steps were 87.1875 for the random agent and 2.0 for the hierarchical RL agent, as printed by this notebook.

9. RESULTS AND DISCUSSION

The evaluation results are summarized below. The hierarchical RL agent achieved a higher success rate than the random navigation baseline in the tested maze. The average successful-step values are reproduced from the notebook output; the random value is averaged over successful random trials, while the RL value is calculated over successful RL trials.

Metric	Random agent	Hierarchical RL agent
Evaluation attempts	100	100
Successful attempts	32	100
Success rate	32%	100%
Average successful steps	87.1875	2.0

10. LIMITATIONS

The experiment uses one small, fixed 5 × 5 maze. The robot is simulated rather than physical. Worker policies are trained for specific start/target pairs, so they may not transfer directly to different maze layouts. The results do not establish generalization to unseen mazes.

11. ETHICAL AND SAFETY CONSIDERATIONS

A real robot should be tested carefully before deployment. It should avoid collisions with people and objects, respect restricted areas and support a safe-stop mechanism. Simulation results alone are not sufficient evidence of real-world safety.

12. CONCLUSION AND FUTURE WORK

This project demonstrates a manager-worker navigation structure using Q-learning. The manager selects a navigation option, and a worker policy chooses individual movements. In the tested maze, the hierarchical RL agent achieved 100% success compared with 32% for the random baseline.

Future work may include testing larger and changing mazes, adding moving obstacles, evaluating multiple unseen layouts, and exploring algorithms such as Deep Q-Networks (DQN) or Proximal Policy Optimization (PPO).

13. REFERENCES

Sutton, R. S. and Barto, A. G., *Reinforcement Learning: An Introduction*, 2nd Edition, MIT Press, 2018.

Watkins, C. J. C. H. and Dayan, P., “Q-learning,” *Machine Learning*, 8, 279–292, 1992.

Python documentation: https://docs.python.org/3/ ; NumPy documentation: https://numpy.org/doc/ ; Matplotlib documentation: https://matplotlib.org/.

14. NOTEBOOK FIGURES AND VISUAL OUTPUTS

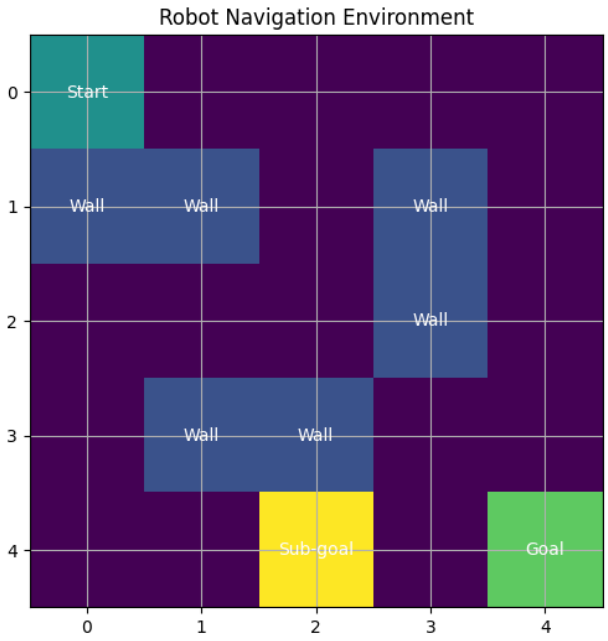


Figure 1. Maze environment created in the notebook.

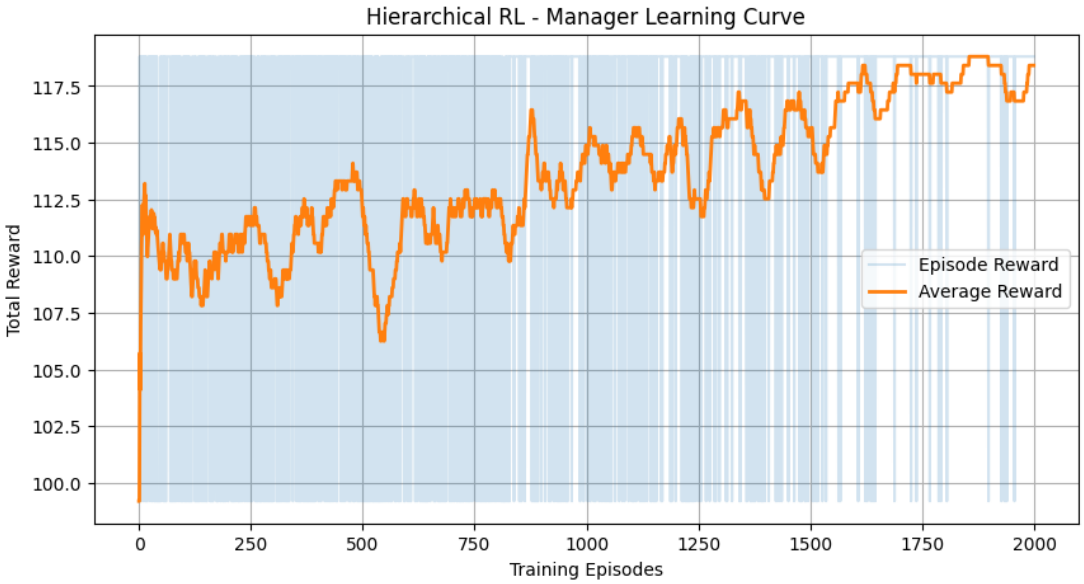


Figure 2. High-level manager learning curve.

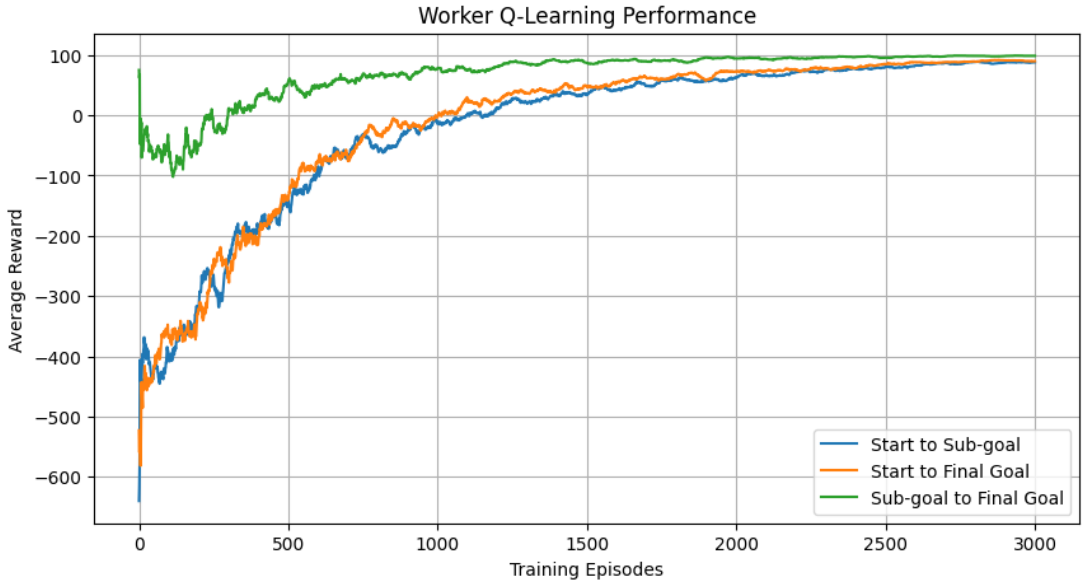


Figure 3. Worker Q-learning performance curves.

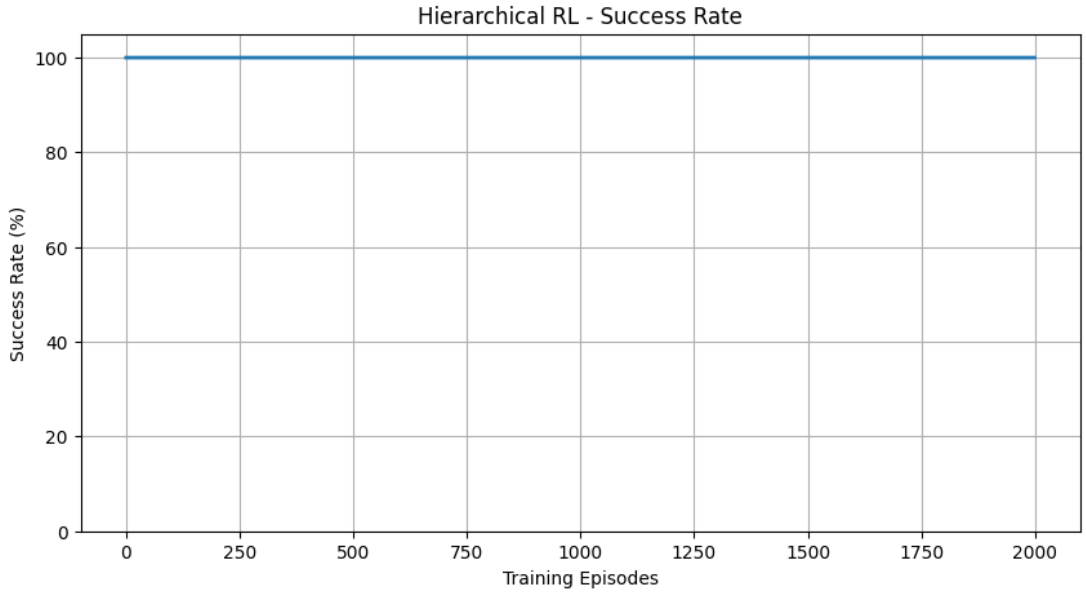


Figure 4. Hierarchical RL manager success-rate curve.

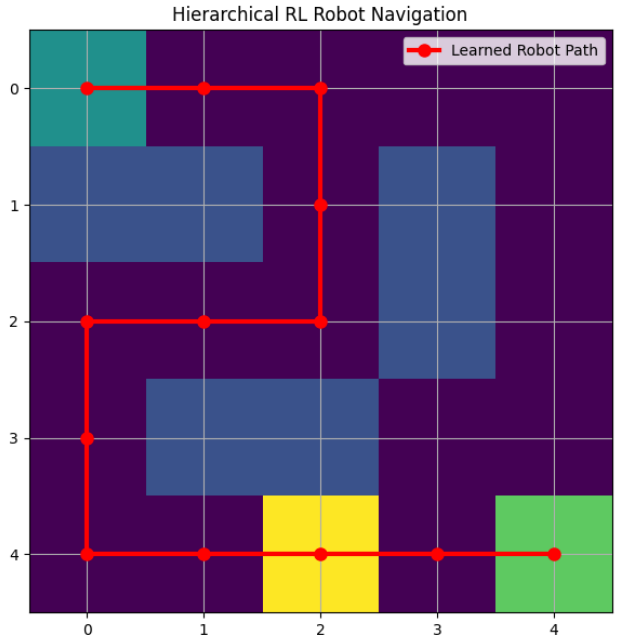


Figure 5. Learned robot path selected by the manager.

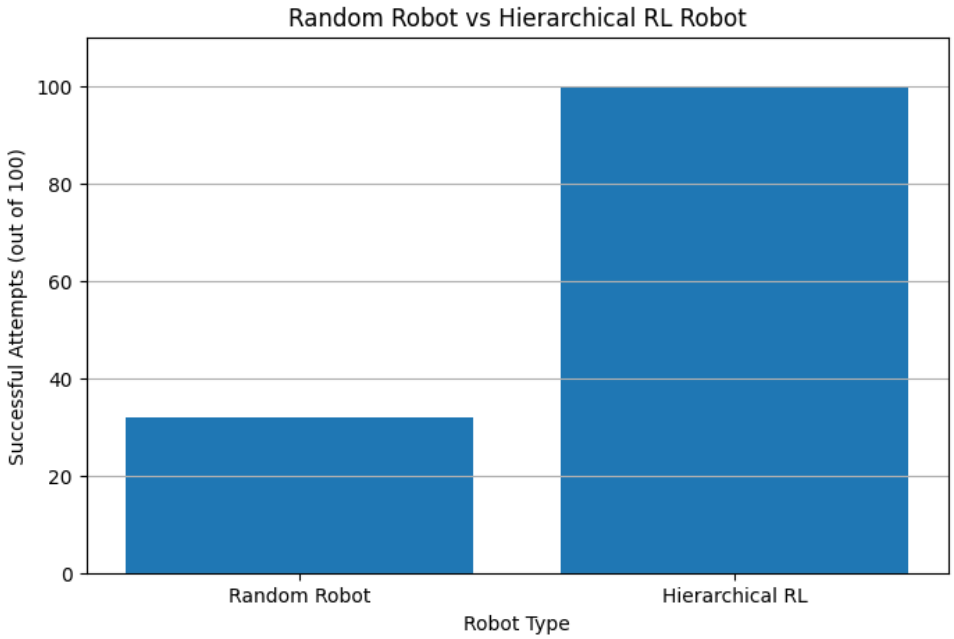


Figure 6. Random robot versus hierarchical RL success comparison.

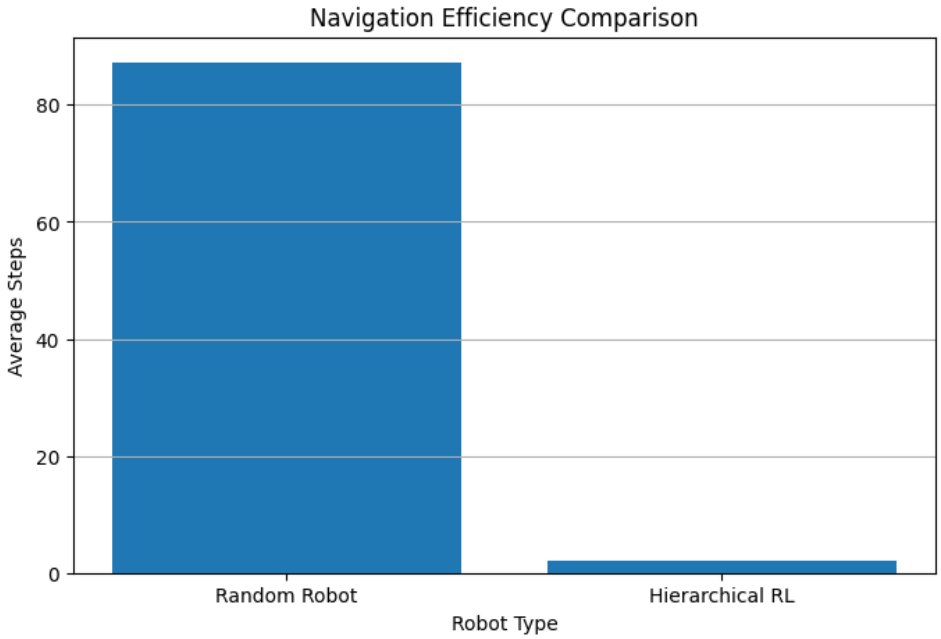


Figure 7. Navigation efficiency comparison using average successful steps.

APPENDIX A — COMPLETE PYTHON NOTEBOOK CODE AND EXECUTION OUTPUTS

The following code cells are reproduced from the uploaded Robot_navigation.ipynb notebook in their original order. Text outputs and displayed figures are included after the corresponding cells where available.

CODE CELL 1

```
import numpy as np
import matplotlib.pyplot as plt
import random

random.seed(42)
np.random.seed(42)

print("All libraries imported successfully!")
```

Execution output — Cell 1

```
All libraries imported successfully!
```

CODE CELL 2

```
# 0 = Empty path
# 1 = Obstacle
# 2 = Start
# 3 = Final Goal
# 4 = Sub-goal

maze = np.array([
    [2, 0, 0, 0, 0],
    [1, 1, 0, 1, 0],
    [0, 0, 0, 1, 0],
    [0, 1, 1, 0, 0],
    [0, 0, 4, 0, 3]
])

ROWS, COLS = maze.shape

START = (0, 0)
SUB_GOAL = (4, 2)
FINAL_GOAL = (4, 4)

ACTIONS = {
    0: (-1, 0),  # Up
    1: (1, 0),   # Down
    2: (0, -1),  # Left
    3: (0, 1)    # Right
}

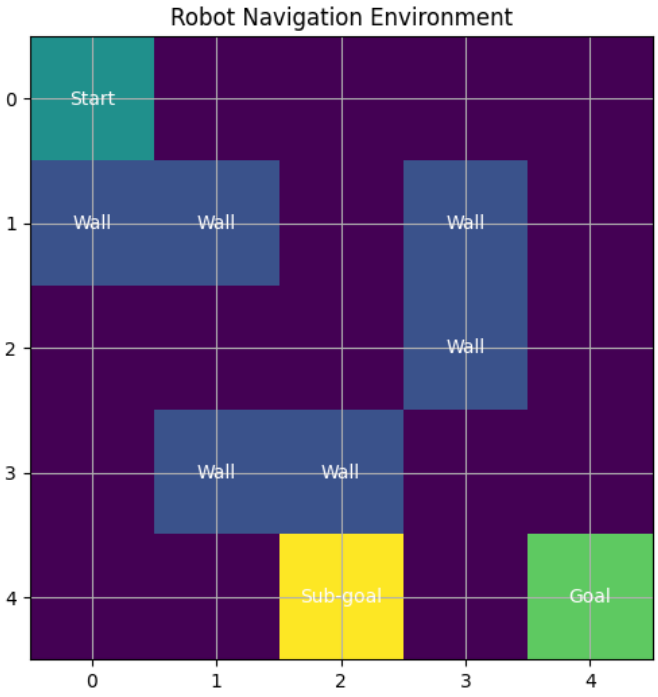
plt.figure(figsize=(6, 6))
plt.imshow(maze, cmap="viridis")

labels = {
    0: "",
    1: "Wall",
    2: "Start",
    3: "Goal",
    4: "Sub-goal"
}

for i in range(ROWS):
    for j in range(COLS):
        plt.text(j, i, labels[maze[i, j]],
                 ha="center", va="center", color="white")

plt.title("Robot Navigation Environment")
plt.xticks(range(COLS))
plt.yticks(range(ROWS))
plt.grid()
plt.show()

print("Maze created successfully!")
```



Execution output — Cell 2

```
Maze created successfully!
```

CODE CELL 3

```
def move_robot(state, action, target):

    dr, dc = ACTIONS[action]

    new_state = (
        state[0] + dr,
        state[1] + dc
    )

    # Check boundaries
    if not (0 <= new_state[0] < ROWS and
            0 <= new_state[1] < COLS):
        return state, -10, False

    # Check obstacles
    if maze[new_state] == 1:
        return state, -10, False

    # Reached destination
    if new_state == target:
        return new_state, 100, True

    # Normal movement
    return new_state, -1, False


print("Robot movement rules created!")
```

Execution output — Cell 3

```
Robot movement rules created!
```

CODE CELL 4

```
ALPHA = 0.1
GAMMA = 0.9
EPISODES = 3000
MAX_STEPS = 100

def train_worker(start, target):

    Q = np.zeros((ROWS, COLS, 4))

    rewards = []
    successes = []

    for episode in range(EPISODES):

        state = start
        total_reward = 0

        epsilon = max(0.05, 1 - episode / EPISODES)

        for step in range(MAX_STEPS):

            # Exploration and exploitation
            if random.random() < epsilon:
                action = random.randint(0, 3)
            else:
                action = np.argmax(Q[state])

            next_state, reward, done = move_robot(
                state, action, target
            )

            # Q-learning update
            old_value = Q[state][action]

            if done:
                next_value = reward
            else:
                next_value = reward + GAMMA * np.max(Q[next_state])

            Q[state][action] = old_value + ALPHA * (
                next_value - old_value
            )

            total_reward += reward
            state = next_state

            if done:
                break

        rewards.append(total_reward)
        successes.append(1 if done else 0)

    return Q, rewards, successes


print("Q-learning worker function created!")
```

Execution output — Cell 4

```
Q-learning worker function created!
```

CODE CELL 5

```
print("Training skill 1: Start to Sub-goal")

Q_start_sub, reward_start_sub, success_start_sub = train_worker(
    START, SUB_GOAL
)

print("Training skill 2: Start to Final Goal")

Q_start_goal, reward_start_goal, success_start_goal = train_worker(
    START, FINAL_GOAL
)

print("Training skill 3: Sub-goal to Final Goal")

Q_sub_goal, reward_sub_goal, success_sub_goal = train_worker(
    SUB_GOAL, FINAL_GOAL
)

print("All worker skills trained successfully!")
```

Execution output — Cell 5

```
Training skill 1: Start to Sub-goal
Training skill 2: Start to Final Goal
Training skill 3: Sub-goal to Final Goal
All worker skills trained successfully!
```

CODE CELL 6

```
# High-level manager Q-table
# State 0 = Start
# State 1 = Sub-goal
# Action 0 = Reach sub-goal
# Action 1 = Reach final goal

manager_Q = np.zeros((2, 2))

def get_path(start, target, Q):

    state = start
    path = [state]

    for _ in range(MAX_STEPS):

        action = np.argmax(Q[state])

        next_state, reward, done = move_robot(
            state, action, target
        )

        if next_state == state:
            return path, False

        state = next_state
        path.append(state)

        if done:
            return path, True

    return path, False


def execute_option(state, action):

    if state == 0 and action == 0:
        return get_path(START, SUB_GOAL, Q_start_sub)

    if state == 0 and action == 1:
        return get_path(START, FINAL_GOAL, Q_start_goal)

    if state == 1 and action == 1:
        return get_path(SUB_GOAL, FINAL_GOAL, Q_sub_goal)

    return [], False


manager_rewards = []
manager_success = []

MANAGER_EPISODES = 2000

for episode in range(MANAGER_EPISODES):

    state = 0
    total_reward = 0
    done = False

    epsilon = max(0.05, 1 - episode / MANAGER_EPISODES)

    while not done:

        if state == 0:
            available_actions = [0, 1]
        else:
            available_actions = [1]

        if random.random() < epsilon:
            action = random.choice(available_actions)
        else:
            action = max(
                available_actions,
                key=lambda a: manager_Q[state, a]
            )

        path, success = execute_option(state, action)

        if not success:
            reward = -50
            next_state = state
            done = True

        elif state == 0 and action == 0:
            # Bonus encourages reaching the intermediate sub-goal
            reward = 20 - 0.1 * (len(path) - 1)
            next_state = 1

        else:
            reward = 100 - 0.1 * (len(path) - 1)
            next_state = state
            done = True

        if done:
```

```
            target_value = reward
        else:
            target_value = reward + GAMMA * max(
                manager_Q[next_state, a]
                for a in ([0, 1] if next_state == 0 else [1])
            )

        manager_Q[state, action] += ALPHA * (
            target_value - manager_Q[state, action]
        )

        total_reward += reward
        state = next_state

    manager_rewards.append(total_reward)
    manager_success.append(1 if total_reward > 0 else 0)

print("High-level manager training completed!")
print("Manager Q-table:")
print(manager_Q)
```

Execution output — Cell 6

```
High-level manager training completed!
Manager Q-table:
[[108.82  99.2 ]
 [  0.    99.8 ]]
```

CODE CELL 7

```
window = 50

average_rewards = [
    np.mean(manager_rewards[max(0, i-window+1):i+1])
    for i in range(len(manager_rewards))
]

plt.figure(figsize=(10, 5))

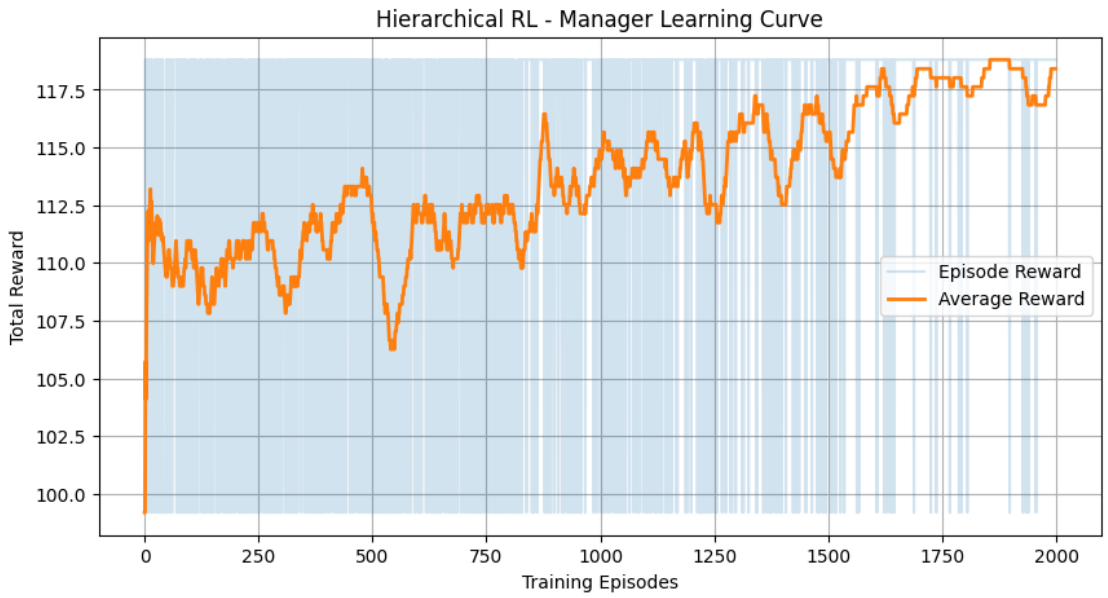
plt.plot(manager_rewards, alpha=0.2,
         label="Episode Reward")

plt.plot(average_rewards, linewidth=2,
         label="Average Reward")

plt.xlabel("Training Episodes")
plt.ylabel("Total Reward")
plt.title("Hierarchical RL - Manager Learning Curve")

plt.legend()
plt.grid()

plt.savefig("manager_learning_curve.png", dpi=300)
plt.show()
```



CODE CELL 8

```
def moving_average(data, window=50):
    return [
        np.mean(data[max(0, i-window+1):i+1])
        for i in range(len(data))
    ]

plt.figure(figsize=(10, 5))

plt.plot(moving_average(reward_start_sub),
         label="Start to Sub-goal")

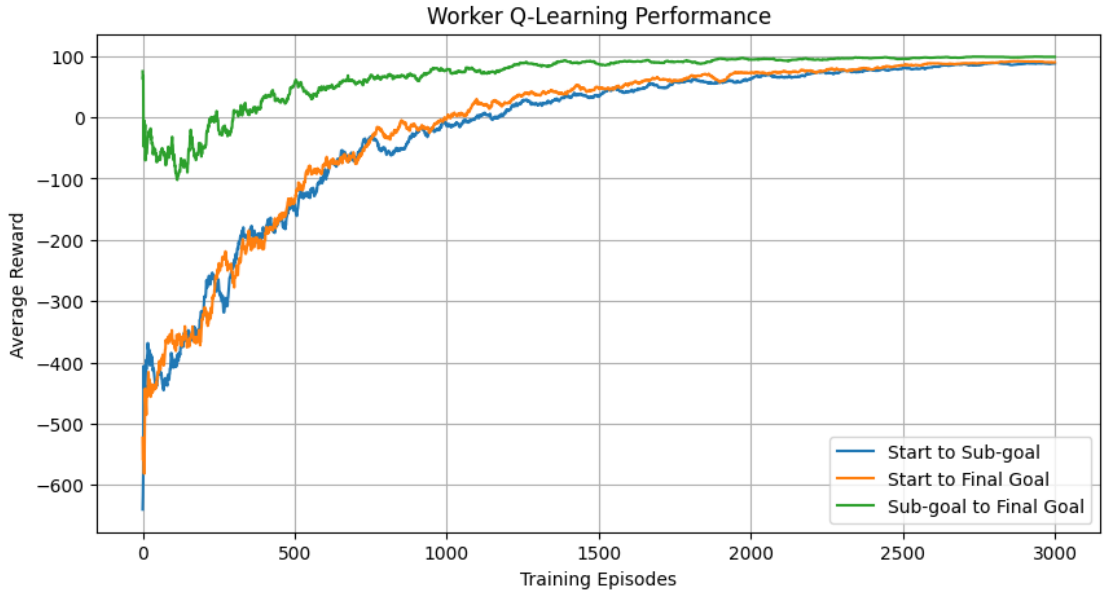
plt.plot(moving_average(reward_start_goal),
         label="Start to Final Goal")

plt.plot(moving_average(reward_sub_goal),
         label="Sub-goal to Final Goal")

plt.xlabel("Training Episodes")
plt.ylabel("Average Reward")
plt.title("Worker Q-Learning Performance")

plt.legend()
plt.grid()

plt.savefig("worker_learning_curve.png", dpi=300)
plt.show()
```



CODE CELL 9

```
success_rate = [
    np.mean(manager_success[max(0, i-49):i+1]) * 100
    for i in range(len(manager_success))
]

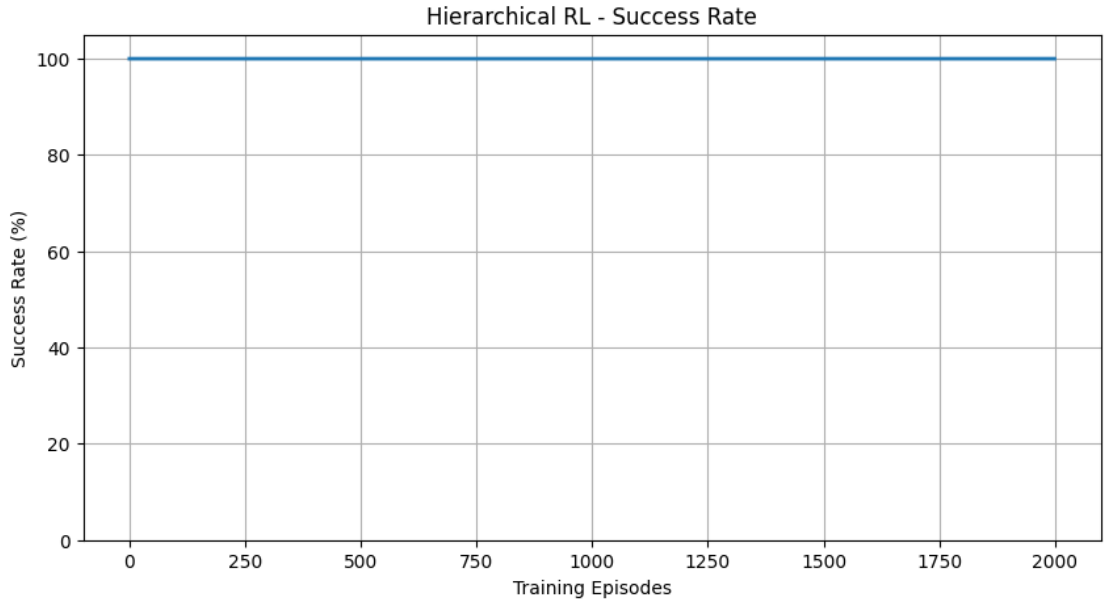
plt.figure(figsize=(10, 5))

plt.plot(success_rate, linewidth=2)

plt.xlabel("Training Episodes")
plt.ylabel("Success Rate (%)")
plt.title("Hierarchical RL - Success Rate")

plt.ylim(0, 105)
plt.grid()

plt.savefig("manager_success_rate.png", dpi=300)
plt.show()
```



CODE CELL 10

```
# Manager selects the best option
selected_option = np.argmax(manager_Q[0])

print("Manager selected option:", selected_option)

if selected_option == 0:

    print("Manager: Reach sub-goal first.")

    path1, success1 = get_path(
        START, SUB_GOAL, Q_start_sub
    )

    if success1:
        path2, success2 = get_path(
            SUB_GOAL, FINAL_GOAL, Q_sub_goal
        )
        robot_path = path1 + path2[1:] if success2 else path1
    else:
        success2 = False
        robot_path = path1

else:

    print("Manager: Go directly to final goal.")

    robot_path, success2 = get_path(
        START, FINAL_GOAL, Q_start_goal
    )

print("Robot reached destination:", success2)

# Visualize path
plt.figure(figsize=(7, 7))

plt.imshow(maze, cmap="viridis")

if robot_path:
    rows = [p[0] for p in robot_path]
    cols = [p[1] for p in robot_path]

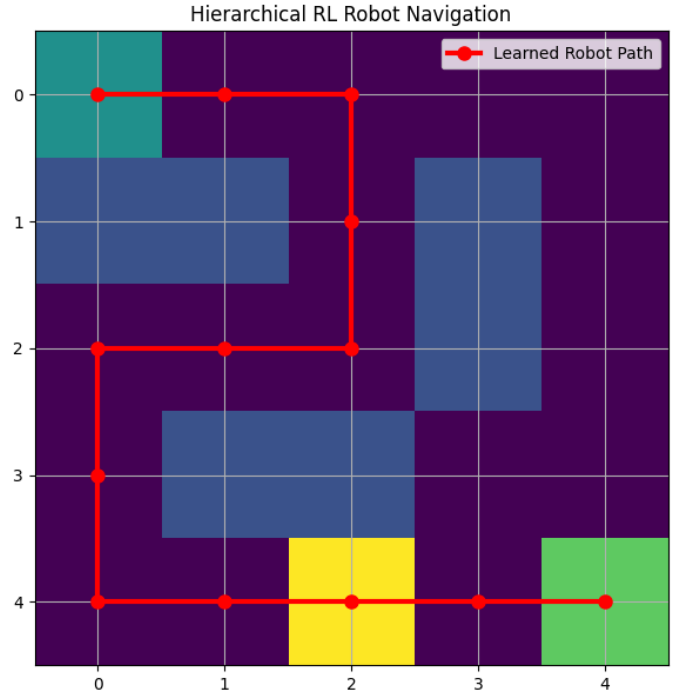
    plt.plot(cols, rows, "r-o",
             linewidth=3, markersize=8,
             label="Learned Robot Path")

plt.title("Hierarchical RL Robot Navigation")
plt.xticks(range(COLS))
plt.yticks(range(ROWS))
plt.grid()
plt.legend()

plt.savefig("robot_navigation_path.png", dpi=300)
plt.show()
```

Execution output — Cell 10

```
Manager selected option: 0
Manager: Reach sub-goal first.
Robot reached destination: True
```


Hierarchical RL Robot Navigation
Learned Robot Path
0
1
2
3
4
0
1
2
3
4

CODE CELL 11

```
# Random robot navigation
def random_navigation(start, target):

    state = start

    for step in range(MAX_STEPS):

        action = random.randint(0, 3)

        next_state, reward, done = move_robot(
            state, action, target
        )

        state = next_state

        if done:
            return True, step + 1

    return False, MAX_STEPS


# Test random robot
random_successes = 0
random_total_steps = 0
random_success_steps = []

for trial in range(100):

    success1, steps1 = random_navigation(
        START, SUB_GOAL
    )

    if success1:

        success2, steps2 = random_navigation(
            SUB_GOAL, FINAL_GOAL
        )

        if success2:
            random_successes += 1
            random_total_steps += steps1 + steps2
            random_success_steps.append(steps1 + steps2)


# Test trained hierarchical robot
rl_successes = 0
rl_total_steps = 0

for trial in range(100):

    option = np.argmax(manager_Q[0])

    if option == 0:

        success1, steps1 = get_path(
            START, SUB_GOAL, Q_start_sub
        )

        if success1:
            success2, steps2 = get_path(
                SUB_GOAL, FINAL_GOAL, Q_sub_goal
            )
        else:
            success2 = False
            steps2 = 0

    else:

        success2, steps2 = get_path(
            START, FINAL_GOAL, Q_start_goal
        )
        success1 = success2
        steps1 = 0

    if success1 and success2:
        rl_successes += 1
        rl_total_steps += steps1 + steps2


# Calculate percentages
random_success_rate = random_successes
rl_success_rate = rl_successes

random_avg_steps = (
    np.mean(random_success_steps)
    if random_success_steps else 0
)

rl_avg_steps = (
    rl_total_steps / rl_successes
    if rl_successes else 0
)
```

```
print("RANDOM ROBOT RESULTS")
print("Successful attempts:", random_successes, "/100")
print("Average successful steps:", random_avg_steps)

print("\nHIERARCHICAL RL ROBOT RESULTS")
print("Successful attempts:", rl_successes, "/100")
print("Average successful steps:", rl_avg_steps)
```

Execution output — Cell 11

```
RANDOM ROBOT RESULTS
Successful attempts: 32 /100
Average successful steps: 87.1875

HIERARCHICAL RL ROBOT RESULTS
Successful attempts: 100 /100
Average successful steps: 2.0
```

CODE CELL 12

```
labels = ["Random Robot", "Hierarchical RL"]

success_values = [
    random_success_rate,
    rl_success_rate
]

plt.figure(figsize=(8, 5))

plt.bar(labels, success_values)

plt.xlabel("Robot Type")
plt.ylabel("Successful Attempts (out of 100)")
plt.title("Random Robot vs Hierarchical RL Robot")

plt.ylim(0, 110)
plt.grid(axis="y")

plt.savefig("random_vs_rl_success.png", dpi=300)
plt.show()


# Average steps comparison
step_values = [
    random_avg_steps,
    rl_avg_steps
]

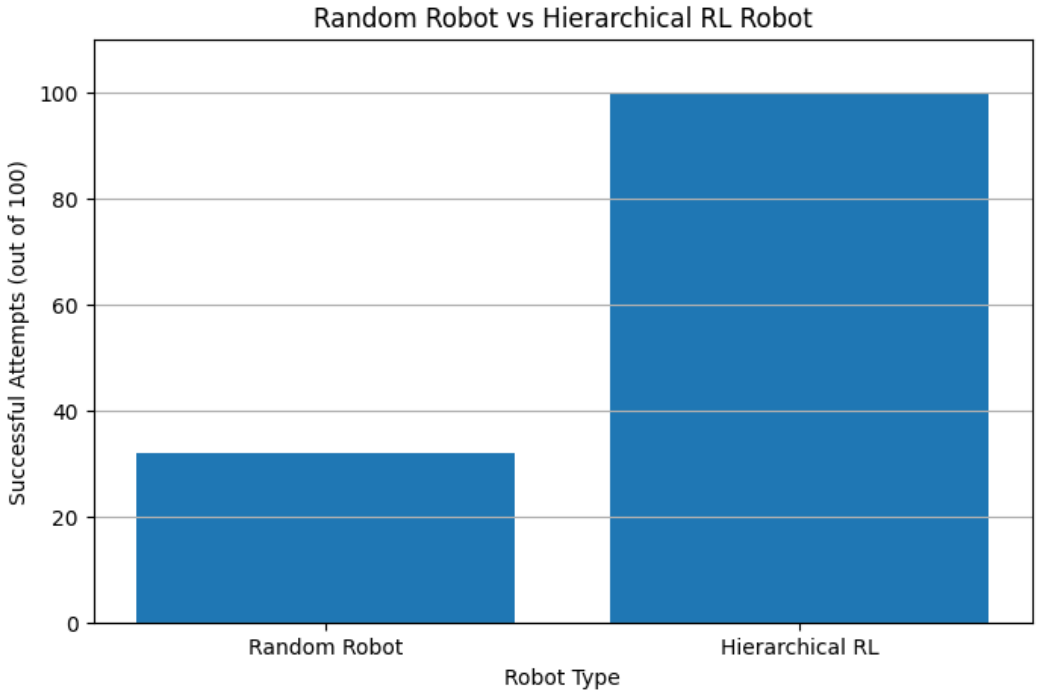
plt.figure(figsize=(8, 5))

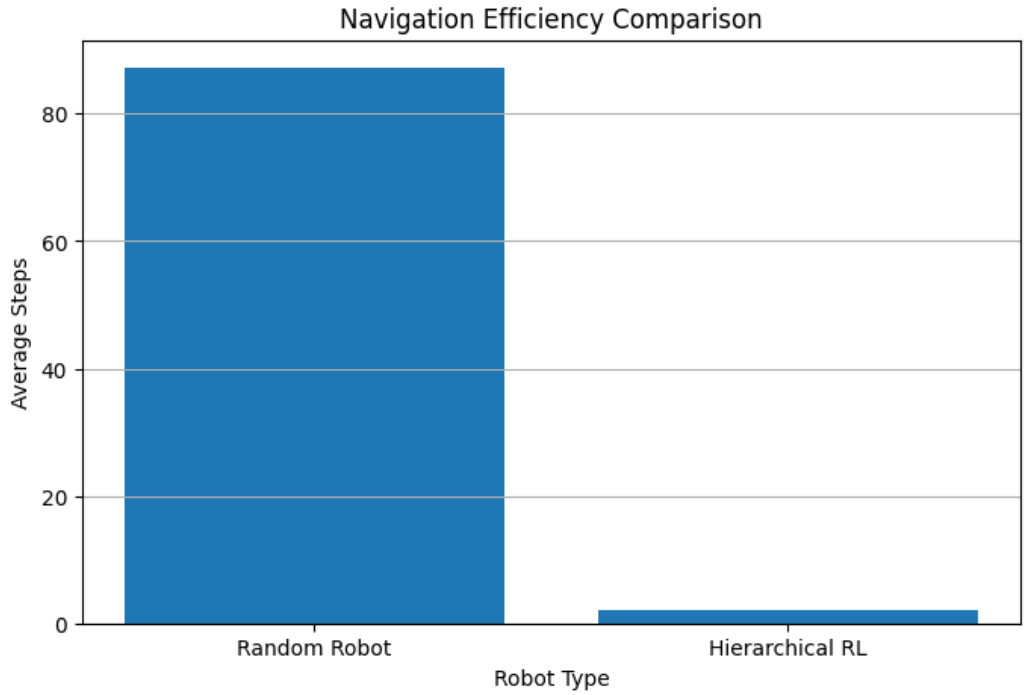
plt.bar(labels, step_values)

plt.xlabel("Robot Type")
plt.ylabel("Average Steps")
plt.title("Navigation Efficiency Comparison")

plt.grid(axis="y")

plt.savefig("random_vs_rl_steps.png", dpi=300)
plt.show()
```





CODE CELL 13

```
print("=" * 45)
print("HIERARCHICAL REINFORCEMENT LEARNING PROJECT")
print("=" * 45)

print("Environment: 5 x 5 Grid Maze")
print("Algorithm: Q-learning")
print("Approach: Hierarchical Reinforcement Learning")

print("\nNavigation Components:")
print("1. High-level Manager")
print("2. Low-level Worker")
print("3. Intermediate Sub-goal")
print("4. Final Destination")

print("\nPerformance:")
print("Random Robot Success Rate:",
      random_success_rate, "%")

print("RL Robot Success Rate:",
      rl_success_rate, "%")

print("\nProject execution completed!")
```

Execution output — Cell 13

```
==========================================
HIERARCHICAL REINFORCEMENT LEARNING PROJECT
==========================================
Environment: 5 x 5 Grid Maze
Algorithm: Q-learning
Approach: Hierarchical Reinforcement Learning

Navigation Components:
1. High-level Manager
2. Low-level Worker
3. Intermediate Sub-goal
4. Final Destination

Performance:
Random Robot Success Rate: 32 %
RL Robot Success Rate: 100 %

Project execution completed!
```

CODE CELL 14

```
import os
import zipfile

graph_files = [
    "manager_learning_curve.png",
    "worker_learning_curve.png",
    "manager_success_rate.png",
    "robot_navigation_path.png",
    "random_vs_rl_success.png",
    "random_vs_rl_steps.png"
]

with zipfile.ZipFile("RL_Project_Graphs.zip", "w") as zip_file:

    for file in graph_files:
        if os.path.exists(file):
            zip_file.write(file)

from google.colab import files
files.download("RL_Project_Graphs.zip")
```

```
<IPython.core.display.Javascript object>
```

```
<IPython.core.display.Javascript object>
```